Module 1 Answers

1. Define Object-Oriented Programming. Mention any three principles of OOP.

   Object-Oriented Programming (OOP) focuses on representing real-world entities as objects that contain data (attributes) and methods (behaviors).

   Three principles of OOP are:

   Encapsulation: It combines data and the methods that operate on that data into a single unit called a class. It also helps protect data by restricting direct access.

   Inheritance: It allows one class to acquire the properties and methods of another class. This promotes code reusability and reduces code duplication.

   Polymorphism: It allows the same method or operation to perform different actions depending on the object or context. This improves the flexibility and extensibility of a program.

2. Differentiate between a class and an object with an example.

Class	Object
A class is a blueprint or template.	An object is an instance of a class.
It defines data and methods.	It uses the data and methods of the class.
No memory is allocated for a class declaration.	Memory is allocated when an object is created.

```
class Student {
        String name;
    }
Student s = new Student();
```

3. What is encapsulation? How is it achieved in Java?

   Encapsulation is the process of wrapping data and methods into a single unit, called a class. Data can be protected using access modifiers such as

   private → strongest data hiding

   default / package-private → accessible only within the same package

   protected → accessible within the same package and by subclasses

   public → accessible everywhere

4. Differentiate between abstraction and encapsulation.

Abstraction	Encapsulation
Hides unnecessary implementation details.	Wraps data and methods into a single unit.
Focuses on **what an object does**.	Focuses on **how data is protected**.
Achieved using abstract classes and interfaces.	Achieved using classes and access specifiers.

5. Define polymorphism. Mention its different types.

Polymorphism means one name having many forms.

The two main types are:

Compile-time polymorphism – achieved using method overloading.

Runtime polymorphism – achieved using method overriding.

6. What is inheritance? Mention the advantages of inheritance.

Inheritance is a mechanism in Java that allows one class (child/subclass) to inherit the properties and methods of another class (parent/superclass).

It is implemented using the extends keyword

```
class Animal {
    void eat() {
        System.out.println("Animal eats");
    }
}

class Dog extends Animal {
    void bark() {
        System.out.println("Dog barks");
    }
}

public class Main {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.eat();
        d.bark();
    }
}
```

```
Animal eats
Dog barks
```

Advantages:

Code reusability

Reduces code duplication

Supports hierarchical classification

Supports method overriding and runtime polymorphism

7. What is JDK? List the main components of JDK.

The Java Development Kit (JDK) is a software package that provides the tools required to develop, compile, debug, and run Java applications.

Main components:

JRE (Java Runtime Environment)

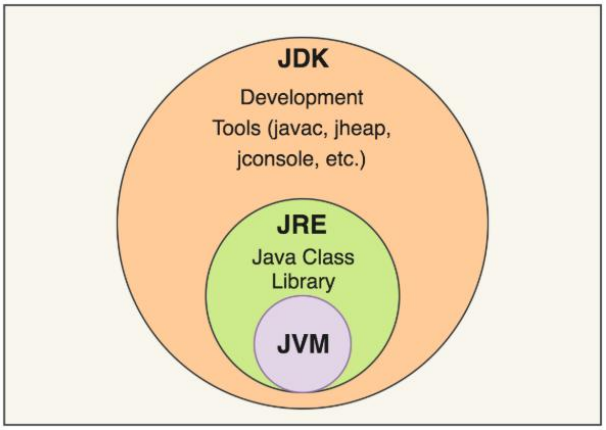
JVM (Java Virtual Machine)

Java compiler (javac)

Java development tools and libraries

8. Differentiate between JDK, JRE and JVM.

JDK	JRE	JVM
Used to develop and run Java programs.	Used to run Java programs.	Executes Java bytecode.
Contains JRE and development tools.	Contains JVM and libraries.	Part of JRE.
Includes compiler.	Does not include compiler.	Does not compile source code.



9. Classify the primitive data types available in Java with examples.

Category	Data Types
Integer	byte, short, int, long
Floating point	float, double
Character	char
Boolean	boolean

```
int age = 20;
double salary = 25000.5;
char grade = 'A';
boolean result = true;
```

10. Differentiate between instance variables, local variables and static variables.

Instance Variable	Local Variable	Static Variable
Declared inside a class but outside methods.	Declared inside a method or block.	Declared using static.
Each object has its own copy.	Exists only during method execution.	Shared by all objects.
Belongs to an object.	Belongs to a method.	Belongs to the class.

```
class VariableDemo {
// Instance variable
int instanceVar = 10;
// Static variable
static int staticVar = 20;
void display() {
// Local variable
int localVar = 30;
System.out.println("Instance Variable: " + instanceVar);
System.out.println("Static Variable: " + staticVar);
System.out.println("Local Variable: " + localVar);
}
public static void main(String[] args) {
VariableDemo obj = new VariableDemo();
obj.display();
}
}
Instance Variable: 10
Static Variable: 20
Local Variable: 30
```

11. What is an array? Write the syntax for declaring and initializing a one-dimensional array.

An array is a collection of elements of the same data type stored using a single name.

Declaration: int[] a;

Initialization: int[] a = {10, 20, 30, 40, 50};

```
public class Main {
    public static void main(String[] args) {
        int[] arr = {10, 20, 30, 40, 50};
        System.out.println("Array Elements:");
        for (int i = 0; i < arr.length; i++) {
            System.out.println(arr[i]);
        }
    }
}
```

12. Differentiate between one-dimensional and two-dimensional arrays with suitable examples.

One-Dimensional Array	Two-Dimensional Array
Stores elements in a single row.	Stores elements in rows and columns.
Uses one index.	Uses two indices.

```
int[] a = {10, 20, 30};

int[][] b = {
    {1, 2},
    {3, 4}
};
```

13. List any six categories of operators available in Java.

Category	Example
1. Arithmetic operators	a + b
2. Relational operators	a > b
3. Logical operators	(a > b) && (b > c)
4. Assignment operators	a += 5
5. Unary operators	a++
6. Bitwise operators	a & b

14. Differentiate between == and .equals() in Java.

==	.equals()
Compares object references.	Compares the content of objects.
Checks whether references point to the same object.	Checks logical equality.

```
String s1 = new String("Java");
String s2 = new String("Java");
System.out.println(s1 == s2);        // false
System.out.println(s1.equals(s2));   // true
```

15. Differentiate between break and continue statements.

break	continue
Terminates the loop completely.	Skips the current iteration.
Control moves outside the loop.	Control moves to the next iteration.

```
class BreakContinueDemo {
    public static void main(String[] args) {
        System.out.println("Using break:");
        for (int i = 1; i <= 5; i++) {
            if (i == 3)
                break;
            System.out.println(i);
        }
        System.out.println("Using continue:");
        for (int i = 1; i <= 5; i++) {
            if (i == 3)
                continue;
            System.out.println(i);
        }
    }
}
```

16. Compare while and do-while loops.

while	do-while
Entry-controlled loop.	Exit-controlled loop.
Condition is checked first.	Condition is checked after execution.
May execute zero times.	Executes at least once.

17. Define a class and write the syntax for creating an object in Java.

A class is a user-defined blueprint used to create objects. It contains data members and methods.

```
class Student {
    String name;
    int rollNo;

    void display() {
        System.out.println("Name: " + name);
        System.out.println("Roll No: " + rollNo);
    }
}

public class Main {
    public static void main(String[] args) {
        Student s1 = new Student(); // Creating an object
        s1.name = "Rahul";
        s1.rollNo = 101;
        s1.display();
    }
}
```

18. What is a constructor? List any three characteristics of a constructor.

A constructor is a special method used to initialize objects.

Characteristics:

- Constructor name must be the same as the class name.
- It has no return type.
- It is called automatically when an object is created.
- It is used to initialize instance variables.
- Constructors can be overloaded.
- Constructors cannot be inherited.
- Constructors cannot be static, final, or abstract.

```
class Student {
    Student() {
        System.out.println("Object created");
    }
}
```

19. Differentiate between a default constructor and a parameterized constructor.

Default/No-Argument Constructor	Parameterized Constructor
Has no parameters	Has one or more parameters.
Initializes objects with default or fixed values.	Initializes objects with specified values.

```
class Student {
    String name;
    int age;
    // Default constructor
    Student() {
        name = "Unknown";
        age = 0;
    }
    // Parameterized constructor
    Student(String n, int a) {
        name = n;
        age = a;
    }
    void display() {
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
    }
    public static void main(String[] args) {
        // Calling default constructor
        Student s1 = new Student();
        s1.display();
        // Calling parameterized constructor
        Student s2 = new Student("Raja", 22);
        s2.display();
    }
}
```

```
Name: Unknown
Age: 0
Name: Raja
Age: 22
```

20. What is method overriding? State the conditions required for method overriding.

Method overriding occurs when a subclass provides its own implementation of a method already defined in the superclass.

Conditions:

The method must have the same name.

It must have the same parameter list.

It requires an inheritance relationship.

The return type must be compatible.

```
class Animal {
    void sound() {
        System.out.println("Animal makes a sound");
    }
}
class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Dog barks");
    }
}
public class Main {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.sound();
    }
}

Dog barks
```

21. What are access specifiers in Java? List the different access specifiers.

Access specifiers control the visibility and accessibility of classes, variables, methods, and constructors.

The four access levels are:

Access Specifier	Description
private	Accessible only within the same class.
Default (no keyword)	Accessible only within the same package.
protected	Accessible within the same package and by subclasses in other packages.
public	Accessible from anywhere in the program.

22. Differentiate between private, default, protected and public access specifiers.

Access Specifier	Description
private	Accessible only within the same class.
Default (no keyword)	Accessible only within the same package.
protected	Accessible within the same package and by subclasses in other packages.
public	Accessible from anywhere in the program.

Access Specifier	Same Class	Same Package	Subclass	Other Package
private	Yes	No	No	No
Default	Yes	Yes	No	No
protected	Yes	Yes	Yes	Yes*
public	Yes	Yes	Yes	Yes

23. What is a static variable? How is it different from an instance variable?

A static variable is also called as class variable.

- Shared by all objects of the class.
- Memory is allocated only once when the class is loaded.
- Accessed using the class name.

```
class Student {
    static String college = "ABC College";
    String name;

    Student(String name) {
        this.name = name;
    }
}

public class Main {
    public static void main(String[] args) {
        Student s1 = new Student("John");
        Student s2 = new Student("Alice");

        System.out.println(Student.college);
        System.out.println(s1.college);
        System.out.println(s2.college);

    }
}
ABC College
ABC College
ABC College
```

24. What is a static method? Can a static method access instance variables directly? Justify your answer.

Static Methods

- Belong to the class.
- Can be called without creating an object.
- Can access only static variables and static methods directly.
- Cannot use this or super.

A static method **cannot directly access instance variables** because instance variables belong to objects, while a static method belongs to the class.

An object reference is required to access instance members.

25. List the different types of inheritance supported in Java. Why does Java not support multiple inheritances through classes?

The types of inheritance are:

Single inheritance

Multilevel inheritance

Hierarchical inheritance

Java does not support multiple inheritances through classes to avoid ambiguity. However, Java can achieve multiple inheritances through interfaces.

```
interface A { }
interface B { }
class C implements A, B { }
```

26. What is constructor chaining?

Constructor chaining means one constructor calls another using this().

```
class Demo {
   Demo() {
      this(10);
      System.out.println("No-Argument Constructor");
   }
   Demo(int x) {
```

```
        this(10, 20);
        System.out.println("One-Argument Constructor");
    }
    Demo(int x, int y) {
        System.out.println("Two-Argument Constructor");
    }
}
public class Main {
    public static void main(String[] args) {
        Demo d = new Demo();
    }
}
```

```
Two-Argument Constructor
One-Argument Constructor
No-Argument Constructor
```

27. List the difference between static variable and instance variable

Static Variable	Instance Variable
Belongs to the class.	Belongs to an object.
One copy is shared by all objects.	Each object has its own copy.
Accessed using the class name.	Accessed using an object.